

# .NET + PostgreSQL Production Guide

Clean Architecture Folder Structure, Connection Pooling & Security Best Practices

## 1. Recommended Folder Structure (Clean Architecture / Onion Architecture)

```
MySolution/
├── src/
│   ├── MyApi.Domain/                # Core Entities, Enums, Value Objects (No Dependencies)
│   │   ├── Entities/
│   │   │   └── User.cs
│   │   ├── Exceptions/
│   │   └── Interfaces/
│   ├── MyApi.Application/          # Business Logic, DTOs, CQRS/Use Cases, Interfaces
│   │   ├── Common/
│   │   │   └── Interfaces/
│   │   ├── Dtos/
│   │   │   └── UserDto.cs
│   │   ├── Features/
│   │   │   └── Users/
│   │   │       ├── Commands/
│   │   │       └── Queries/
│   ├── MyApi.Infrastructure/       # DB Context, PostgreSQL Driver, Repositories, External APIs
│   │   ├── Persistence/
│   │   │   ├── ApplicationDbContext.cs
│   │   │   ├── Migrations/
│   │   │   └── Repositories/
│   │   └── Services/
│   └── MyApi.Web/                  # API Controllers / Minimal APIs, Middleware, Program.cs
│       ├── Controllers/
│       ├── Middlewares/
│       ├── Program.cs
│       └── appsettings.json
├── tests/
│   ├── MyApi.UnitTests/
│   └── MyApi.IntegrationTests/
```

## 2. .NET + PostgreSQL Connection & Driver Setup

- **Npgsql Driver:** Use `Npgsql.EntityFrameworkCore.PostgreSQL` for EF Core or `Npgsql` directly for Dapper.
- **Connection String (appsettings.json):** Always configure pooling parameters explicitly.

```
"ConnectionStrings": {
  "DefaultConnection":
    "Host=localhost;Port=5432;Database=mydb;Username=postgres;Password=secret;Pooling=true;Minimum Pool
    Size=5;Maximum Pool Size=100;Connection Lifetime=300;"
}
```

### 3. .NET Backend Best Practices

---

- **Asynchronous All The Way:** Use `async / await` for all I/O and database operations to keep CLR thread pool threads available.
- **Global Exception Handling:** Utilize custom ASP.NET Core middleware or `ExceptionHandler` (introduced in .NET 8) to convert uncaught exceptions into RFC 7807 `ProblemDetails` JSON responses.
- **Cancellation Tokens:** Pass `CancellationToken` from HTTP controllers down to `DbContext`/`Dapper` methods to cancel database operations if the client aborts the request.
- **Dependency Injection & Lifetimes:** Register `DbContext` as `Scoped`. Register stateless helper services as `Transient` or `Singleton`.
- **Data Transfer Objects (DTOs):** Never expose Domain Entities directly via API responses; map them to lightweight records/DTOs.

### 4. PostgreSQL Integration Best Practices

---

- **Connection Pooling:** Rely on `Npgsql`'s built-in client connection pool or deploy **PgBouncer** in front of PostgreSQL for high-concurrency microservice workloads.
- **Snake\_Case Naming:** PostgreSQL defaults to lowercase identifiers. Enable EF Core `snake_case` naming conventions using `UseSnakeCaseNamingConvention()`.
- **Parameterization:** Always parameterize raw SQL queries (especially in `Dapper`) to prevent SQL Injection and maximize PostgreSQL query plan caching.
- **EF Core Performance:** Apply `.AsNoTracking()` for read-only GET queries to bypass state tracker overhead.